

Machine Learning: Classification and Logistic Regression Optimization

Quoc Kien

1. Bản chất của Bài toán Phân loại (Classification)

Trong bài toán phân loại nhị phân (Binary Classification), tập dữ liệu huấn luyện bao gồm n mẫu có dạng $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$, trong đó nhãn mục tiêu bị giới hạn trong hai giá trị rời rạc:

$$y^{(i)} \in \{0, 1\}$$

- $y^{(i)} = 1$: Lớp dương tính (Positive class).
- $y^{(i)} = 0$: Lớp âm tính (Negative class).

Tại sao Tuyến tính (Linear Regression) thất bại với Phân loại?

Nếu cố chấp sử dụng mô hình tuyến tính $f_{\beta}(\mathbf{x}) = \beta^T \mathbf{x}$ cho tác vụ phân loại, đường ranh giới quyết định sẽ bị ảnh hưởng nghiêm trọng bởi các điểm dữ liệu dị biệt (Outliers). Khi xuất hiện một điểm dữ liệu nằm rất xa nhưng mang nhãn cực đoan, việc tối thiểu hóa bình phương khoảng cách (MSE) sẽ ép đường thẳng tuyến tính phải xoay một góc rất lớn để bù trừ sai số. Hệ quả là đường ranh giới bị dịch chuyển một cách vô lý, trực tiếp gây ra phân loại sai cho các điểm dữ liệu bình thường ở gần.

Do đó, ta cần một hàm kích hoạt có khả năng ép toàn bộ không gian số thực về khoảng xác suất $(0, 1)$, đó chính là **Hàm số Sigmoid**.

2. Mô hình Logistic Regression & Hàm Sigmoid

Logistic Regression mô hình hóa **xác suất có điều kiện** để một mẫu dữ liệu được phân loại vào lớp 1:

$$P(y^{(i)} = 1 \mid \mathbf{x}^{(i)}) = f_{\beta}(\mathbf{x}^{(i)}) = \sigma(\beta^T \mathbf{x}^{(i)})$$

Hàm số kích hoạt Sigmoid $\sigma(z)$ được định nghĩa toán học là:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1)$$

Biểu thức tính xác suất cho từng trường hợp nhãn cụ thể của mẫu dữ liệu: * $P(y^{(i)} = 1 \mid \mathbf{x}^{(i)}; \beta) = f_{\beta}(\mathbf{x}^{(i)})$ * $P(y^{(i)} = 0 \mid \mathbf{x}^{(i)}; \beta) = 1 - f_{\beta}(\mathbf{x}^{(i)})$

Công thức Xác suất Hợp nhất

Để thuận tiện cho việc thiết lập hàm toán học tối ưu, hai biểu thức trên được gộp lại thành một phương trình duy nhất dưới dạng tích lũy thừa nhờ tính chất nhị phân của nhãn $y^{(i)} \in \{0, 1\}$:

$$P(y^{(i)} \mid \mathbf{x}^{(i)}; \beta) = [f_{\beta}(\mathbf{x}^{(i)})]^{y^{(i)}} [1 - f_{\beta}(\mathbf{x}^{(i)})]^{1-y^{(i)}}$$

3. Chứng minh Hàm Mất mát Cross-Entropy bằng MLE

Với giả định các mẫu dữ liệu trong tập huấn luyện độc lập và phân phối cùng quy luật (i.i.d.), hàm lực lượng liên hợp xác suất (Likelihood Function) được thiết lập bằng tích xác suất của tất cả các mẫu:

$$L(\beta) = \prod_{i=1}^n P(y^{(i)} \mid \mathbf{x}^{(i)}; \beta) = \prod_{i=1}^n [f_{\beta}(\mathbf{x}^{(i)})]^{y^{(i)}} [1 - f_{\beta}(\mathbf{x}^{(i)})]^{1-y^{(i)}}$$

Theo nguyên lý ước lượng hợp lý cực đại (Maximum Likelihood Estimation - MLE), ta cần tìm bộ tham số β sao cho giá trị $L(\beta)$ đạt cực đại. Vì hàm logarit là hàm đồng biến đơn điệu, bài toán tương đương với việc tối đa hóa hàm Log-Likelihood:

$$\max_{\beta} L(\beta) \iff \max_{\beta} \log L(\beta) \iff \min_{\beta} -\log L(\beta)$$

Tiến hành lấy logarit tự nhiên và đổi dấu để chuyển sang bài toán tối thiểu hóa hàm mất mát:

$$J(\beta) = -\log L(\beta) = -\sum_{i=1}^n \log \left([f_{\beta}(\mathbf{x}^{(i)})]^{y^{(i)}} [1 - f_{\beta}(\mathbf{x}^{(i)})]^{1-y^{(i)}} \right)$$

Sử dụng tính chất của logarit để đưa số mũ xuống thành hệ số, ta thu được hàm mất mát **Binary Cross-Entropy Loss**:

$$J(\beta) = -\sum_{i=1}^n [y^{(i)} \log f_{\beta}(\mathbf{x}^{(i)}) + (1 - y^{(i)}) \log(1 - f_{\beta}(\mathbf{x}^{(i)}))]$$

4. Chứng minh Tính Lồi (Convex Optimization) bằng Ma trận Hessian

Để các thuật toán tối ưu hóa dựa trên đạo hàm hoạt động tin cậy và không bị kẹt ở các điểm cực tiểu cục bộ, hàm mất mát $J(\beta)$ phải là một hàm lồi. Điều này đồng nghĩa với việc ma trận đạo hàm bậc hai (Hessian Matrix) phải luôn bán xác định dương ($\nabla^2 J(\beta) \succeq 0$) với mọi giá trị của β .

Đạo hàm Bậc một (Gradient) via Chain Rule

Đặt biến trung gian tuyến tính $z^{(i)} = \beta^T \mathbf{x}^{(i)}$ và giá trị dự đoán $\hat{y}^{(i)} = \sigma(z^{(i)}) = f_{\beta}(\mathbf{x}^{(i)})$. Áp dụng quy tắc chuỗi (Chain Rule) để tính đạo hàm riêng cho từng mẫu dữ liệu:

$$\frac{\partial J^{(i)}(\beta)}{\partial \beta} = \frac{\partial J^{(i)}}{\partial \hat{y}^{(i)}} \cdot \frac{\partial \hat{y}^{(i)}}{\partial z^{(i)}} \cdot \frac{\partial z^{(i)}}{\partial \beta}$$

Tính toán chi tiết từng thành phần vi phân:

1. **Thành phần Loss:** $\frac{\partial J^{(i)}}{\partial \hat{y}^{(i)}} = \frac{-y^{(i)}}{\hat{y}^{(i)}} + \frac{1-y^{(i)}}{1-\hat{y}^{(i)}} = \frac{\hat{y}^{(i)} - y^{(i)}}{\hat{y}^{(i)}(1-\hat{y}^{(i)})}$
2. **Thành phần Sigmoid:** $\frac{\partial \hat{y}^{(i)}}{\partial z^{(i)}} = \sigma(z^{(i)})(1 - \sigma(z^{(i)})) = \hat{y}^{(i)}(1 - \hat{y}^{(i)})$
3. **Thành phần Tuyến tính:** $\frac{\partial z^{(i)}}{\partial \beta} = \mathbf{x}^{(i)}$

Nhân kết hợp cả ba thành phần lại với nhau để triệt tiêu đại lượng mẫu số:

$$\frac{\partial J^{(i)}(\beta)}{\partial \beta} = \left[\frac{\hat{y}^{(i)} - y^{(i)}}{\hat{y}^{(i)}(1 - \hat{y}^{(i)})} \right] \cdot [\hat{y}^{(i)}(1 - \hat{y}^{(i)})] \cdot \mathbf{x}^{(i)} = (\hat{y}^{(i)} - y^{(i)}) \mathbf{x}^{(i)}$$

Đạo hàm Bậc hai (Ma trận Hessian)

Tiếp tục lấy đạo hàm bậc hai bằng cách biệt hóa toán tử Gradient thu được theo vector tham số β^T :

$$H = \nabla^2 J(\beta) = \frac{\partial^2 J(\beta)}{\partial \beta \partial \beta^T} = \sum_{i=1}^n \frac{\partial}{\partial \beta} [(\hat{y}^{(i)} - y^{(i)}) \mathbf{x}^{(i)}]$$

Vì nhãn dữ liệu thực tế $y^{(i)}$ là một hằng số cố định và ta đã biết $\frac{\partial \hat{y}^{(i)}}{\partial \beta} = \hat{y}^{(i)}(1 - \hat{y}^{(i)}) \mathbf{x}^{(i)}$, biểu thức ma trận Hessian tổng thể rút gọn thành:

$$H = \sum_{i=1}^n \hat{y}^{(i)}(1 - \hat{y}^{(i)}) \mathbf{x}^{(i)} (\mathbf{x}^{(i)})^T$$

Biện luận tính lồi: * Vì đầu ra hàm xác suất Sigmoid luôn nằm trong biên hạn hẹp $\hat{y}^{(i)} \in (0, 1)$, hệ số trọng số luôn dương: $\hat{y}^{(i)}(1 - \hat{y}^{(i)}) > 0$. * Bản chất phép nhân ngoài của một vector với chính nó dưới dạng $\mathbf{x}\mathbf{x}^T$ luôn tạo dựng nên một ma trận bán xác định dương.

Do tổng các ma trận bán xác định dương là một ma trận bán xác định dương, ta kết luận $H \succeq 0 \forall \beta$. Hàm mục tiêu của Logistic Regression **hoàn toàn là hàm lồi (Convex Function)**.

5. Các Phương pháp Tối ưu hóa Toán học

Thuật toán Gradient Descent (Tối ưu bậc 1)

Thực hiện cập nhật véc-tơ trọng số lặp bằng cách đi ngược hướng với vector độ dốc Gradient hệ số học α :

$$\beta^{(t+1)} = \beta^{(t)} - \alpha \nabla J(\beta^{(t)}) = \beta^{(t)} + \alpha \sum_{i=1}^n (y^{(i)} - f_{\beta}(\mathbf{x}^{(i)})) \mathbf{x}^{(i)}$$

Công thức cập nhật chi tiết cho từng thành phần chỉ số tham số β_j trong không gian đa chiều:

$$\beta_j^{(t+1)} = \beta_j^{(t)} + \alpha \sum_{i=1}^n (y^{(i)} - f_{\beta}(\mathbf{x}^{(i)})) x_j^{(i)}$$

Phương pháp Newton - Raphson (Tối ưu bậc 2)

Để tìm điểm cực trị của hàm lỗi bằng cách giải hệ phương trình đạo hàm bằng không $\nabla J(\beta) = 0$, phương pháp Newton sử dụng khai triển Taylor bậc hai nhằm mượn thêm thông tin độ cong từ ma trận Hessian.

Công thức cập nhật đa biến sử dụng ma trận nghịch đảo Hessian tại mỗi bước lặp:

$$\beta^{(t+1)} = \beta^{(t)} - H^{-1} \nabla J(\beta^{(t)})$$

So sánh đặc tính vận hành:

Thuật toán Tối ưu	Cơ chế Đạo hàm	Ưu điểm Vận hành	Nhược điểm Khuyết góc
Gradient Descent	Chỉ sử dụng bậc một (∇J)	Chi phí tính toán trên mỗi bước lặp rất thấp ($\mathcal{O}(p)$). Phù hợp với không gian dữ liệu lớn chứa hàng triệu thuộc tính p .	Tốc độ hội tụ chậm (Linear Convergence). Tốn nhiều thời gian thử nghiệm để tìm ra hệ số học α phù hợp.
Newton's Method	Kết hợp bậc một và ma trận bậc hai ($H^{-1} \nabla J$)	Tốc độ hội tụ cực kỳ nhanh nhờ đặc tính hội tụ bậc hai (Quadratic Convergence). Số vòng lặp yêu cầu rất ít và không cần tùy chỉnh hyperparameter hệ số học α .	Chi phí tính toán ma trận nghịch đảo Hessian rất lớn ($\mathcal{O}(p^3)$) tại mỗi bước lặp. Hoàn toàn bất khả thi nếu số lượng thuộc tính p lớn.

6. Executable Check: Sigmoid and Decision Boundary

The formulas above become easier to remember if we connect them to two visual facts: the sigmoid turns a real-valued score into a probability, and the threshold $P(y = 1 \mid \mathbf{x}) = 0.5$ becomes a linear decision boundary.

```

import numpy as np
import matplotlib.pyplot as plt

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

np.random.seed(7)
n = 120
X0 = np.random.normal(loc=[-1.2, -0.8], scale=[0.55, 0.55], size=(n // 2, 2))
X1 = np.random.normal(loc=[1.2, 1.0], scale=[0.65, 0.65], size=(n // 2, 2))
X = np.vstack([X0, X1])
y = np.hstack([np.zeros(n // 2), np.ones(n // 2)])
X_design = np.c_[np.ones(n), X]

beta = np.zeros(3)
lr = 0.4
for _ in range(800):
    probs = sigmoid(X_design @ beta)
    grad = X_design.T @ (y - probs) / n
    beta += lr * grad

pred = (sigmoid(X_design @ beta) >= 0.5).astype(int)
accuracy = (pred == y).mean()
print("Learned beta:", np.round(beta, 3))
print("Training accuracy:", round(float(accuracy), 3))

z = np.linspace(-8, 8, 300)
xx, yy = np.meshgrid(np.linspace(-3, 3, 160), np.linspace(-3, 3, 160))
grid_design = np.c_[np.ones(xx.size), xx.ravel(), yy.ravel()]
surface = sigmoid(grid_design @ beta).reshape(xx.shape)

fig, ax = plt.subplots(1, 2, figsize=(9, 3.8))
ax[0].plot(z, sigmoid(z), color="#4C78A8")
ax[0].axhline(0.5, color="black", linestyle="--", linewidth=1)
ax[0].axvline(0.0, color="black", linestyle="--", linewidth=1)
ax[0].set_title("Sigmoid maps score to probability")
ax[0].set_xlabel("linear score z")
ax[0].set_ylabel(" $\sigma(z)$ ")
ax[0].grid(alpha=0.25)

contour = ax[1].contourf(xx, yy, surface, levels=20, cmap="RdBu_r",
    ↪ alpha=0.75)
ax[1].scatter(X0[:, 0], X0[:, 1], label="class 0", edgecolor="black",
    ↪ alpha=0.85)

```

```

ax[1].scatter(X1[:, 0], X1[:, 1], label="class 1", edgecolor="black",
    ↪ alpha=0.85)
ax[1].contour(xx, yy, surface, levels=[0.5], colors="black", linewidths=2)
ax[1].set_title("Learned probability boundary")
ax[1].set_xlabel("$x_1$")
ax[1].set_ylabel("$x_2$")
ax[1].legend()
fig.colorbar(contour, ax=ax[1], label="$P(y=1 \mid x)$")
plt.tight_layout()
plt.show()

```

Learned beta: [-0.101 4.325 3.452]
 Training accuracy: 1.0

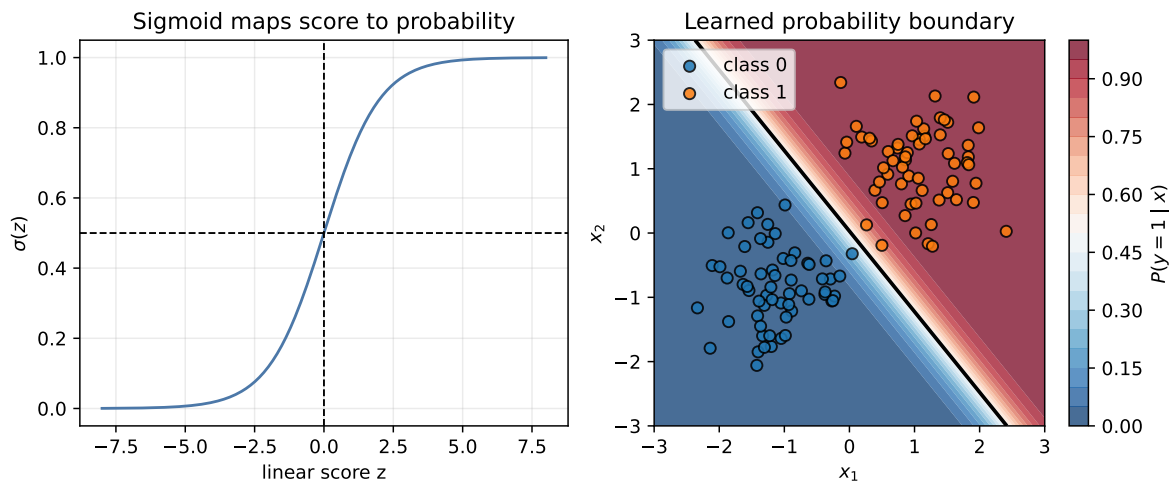


Figure 1: Logistic regression maps a linear score through sigmoid and learns a linear probability boundary.